Beware Cryptographic APIs !

A Case Study with Java.util.Random

Edgard Lima | CISSA | CESAR

- Basic entry level talk
  - Objectives:
    - Spark curiosity
    - Raise awareness
- Requirements
  - Interest
- Helps (not required) to know:
  - A programming language
  - Bitwise operations
  - Math

1: ~/Documents/CISSA

◄ 58s ◎ whoami □ Documents/CISSA 16:13

= = = Past = = =

- 10 years at CESAR (not continous)
- 25 years at Industry/Research
- Consutant at Nokia Research Center (NRC) 2007/2008 - Helsink - Finland
- Author of GStreamer Video4Linux2 SRC
  - Linux WebCam
- Programming since 1995
- Bachelor: Computer Science 2002 - CIn UFPE
- Master: Algorithms and Distributed systems (Distributed van Emde Boas tree)
- Device Driver develpment
- Embedded Systems development
- Customizing Linux kernel for new fresh hardware
- Teacher: Operating Systems and Formal Languages 2017/2018
- Dissecated or developed quite a few network protocols
  - CAN, J1939, Blutooth (physical, l2cap, sdp, rfcomm) ...
- Compilers, sockets, scrapping, IPC, concurrency, etc...
- C, C++, python, Objective-C, Swift, Java
  - It is a messy in my mind (I need an auto-complete now :-)

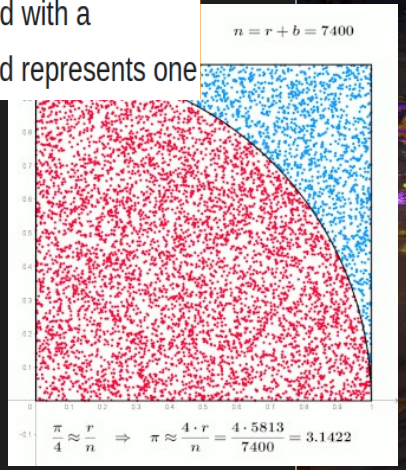
= = = Now = = =

- PhD Student at CESAR School: Cybersecurity
- Researcher at CISSA/CESAR: Cybersecurity
- Dad of two lovely kids, and owner of two fierce Dobermanns

What is a **LCG** ?

WIKIPEDIA
The Free Encyclopedia

A **linear congruential generator** (**LCG**) is an algorithm that yields a sequence of pseudo-randomized numbers calculated with a discontinuous piecewise linear equation. The method represents one



$X[i+1] = (a \cdot X[i] + b) \bmod q$

Output = $X[i+1] / w$

	java.util.Random	python random
Algorithm	LCG	Mersenne Twister
period	2^48	2^19937 - 1
speed	Lower Latency	Higher Throughput
Memory (state)	6 bytes	2.5 KB
Statistical quality	Poor	Excellent
Predictability	Very high (2)	High (624)
Memory (state)	6 bytes	2.5 KB
Use case	IoT	Simulation, Monte Carlo, Games

If you still want to use java.util.random for my Boss Hit game … ok, you are the boss …

```java
public class Player {
    final Random random = new Random();  // per Player
    static final long UINT_RANGE = 0x1_0000_0000L;                          // 2^32
    final double criticalHitChance = 0.60;                                  // Only to show in UI
    final long criticalHitThreshold = (long) (UINT_RANGE * criticalHitChance);  // hold/use this

    Run | Debug
    public static void main(String[] args) {
        Player player = new Player();

        for (int i = 0; i < 5; i++) {
            long hit = player.random.nextInt() & 0xFFFF_FFFFL;  // Simulate a 32-bit unsigned int roll

            if (hit < player.criticalHitThreshold) {  // 0 ≤ roll < threshold
                System.out.printf("Critical Hit! (Roll: %08X)%n", hit);
            } else {
                System.out.printf("Normal Hit. (Roll: %08X)%n", hit);
            }
        }
    }
}
```

Avoid lower bits
Avoid % (mod) nextInt(100)
0.5999999999767169356 = 0.6 = 0.60000000001

java.util.Random

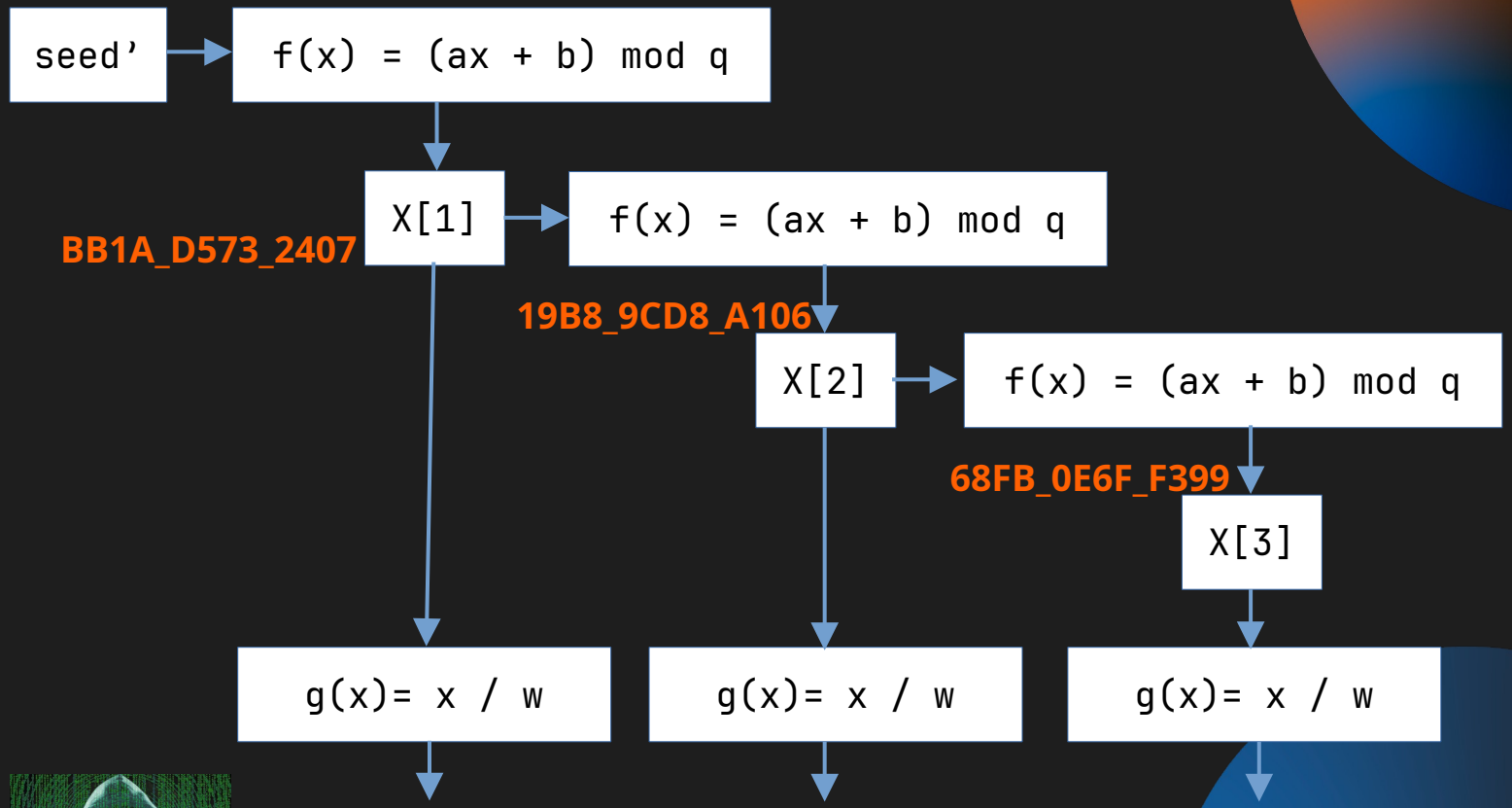
```
X[i+1] = (a · X[i] + b) mod 2^48 =
     (a · X[i] + b) & 0xFFFF_FFFF_FFFF

Output = X[i+1] / 2^16
= X[i+1] >>> 16
Output the 32 top most significant bits

i.e. hides 16 less significant bits
```

a	25214903917 = 0x5DEECE66D
b	11 = 0xB
m	2^48=281.474.976.710.656 = 0x1_0000_0000_0000
m-1	(2^48)-1= 281.474.976.710.655 = 0x0_FFFF_FFFF_FFFF

🧠 **The Hull–Dobell theorem**

An LCG has **full period** if and only if all three conditions hold:

1. c and m are **coprime**
2. $a - 1$ is divisible by **all prime factors of** m
3. If m is divisible by 4, then $a - 1$ is also divisible by 4


seed’
f(x) = (ax + b) mod q
X[1]
BB1A_D573_2407
f(x) = (ax + b) mod q
19B8_9CD8_A106
X[2]
f(x) = (ax + b) mod q
68FB_0E6F_F399
X[3]
g(x)= x / w
g(x)= x / w
g(x)= x / w
BB1A_D573
19B8_9CD8
68FB_0E6F

Cryptanalysis

From Wikipedia, the free encyclopedia

Cryptanalysis (from the Greek *kryptós*, "hidden", and *analýein*, "to analyze") refers to the process of analyzing information systems in order to understand hidden aspects of the systems.[1] Cryptanalysis is used to breach cryptographic security systems and gain access to the contents of encrypted messages, even if the cryptographic key is unknown.

Reconstruction of the appearance of cyclometer, a device used to break the encryption of an early version of the Enigma machine. Based on sketches in Marian Rejewski's memoirs.

In addition to mathematical analysis of cryptographic algorithms, cryptanalysis includes the study of side-channel attacks that do not target weaknesses in the cryptographic algorithms themselves, but instead exploit weaknesses in their implementation.

Even though the goal has been the same, the methods and techniques of cryptanalysis have changed drastically through the history of cryptography, adapting to increasing cryptographic

```cpp
static uint64_t calculateLcgSeedByBruteForceT(const std::
    const uint64_t w = (1ULL << t); // e.g. 2^16 = 65536

    uint64_t state = 0;

    if (values.size() < 2) {
        throw std::invalid_argument("Need at least one pa
    }

    uint64_t s0 = values[0];
    for (uint64_t lower = 0; lower < w; ++lower) {
        uint64_t s0_full = s0 << t;
        s0_full |= lower;
        uint64_t s1_full = (a * s0_full + b) & q_mask;
        uint64_t s1 = s1_full >> t;

        if (s1 == values[1]) {
            uint64_t si_full = s1_full;
            uint64_t sj_full = 0;
            size_t i = 1;
            for (; i < values.size() - 1; ++i) {
                sj_full = (a * si_full + b) & q_mask;
                uint64_t sj = sj_full >> t;

                if (sj != values[i + 1]) {
                    break;
                } else {
                    si_full = sj_full;
                }
            }

            if (i == values.size() - 1) {
                state = sj_full;
                goto Zer0Leak;
            }
        }
    }

    throw std::runtime_error("Seed not found!");

Zer0Leak:
    return state;
}
```

https://github.com/Zer0Leak/CryptoPlayground

```
1: .../BreakingLCG/build/Release

./breaking_lcg
  build/Release C++ v18.1.3-clang++ v3.28.3   23:57
Input values:
#########################
17 03 0D
0D 06 01
0A 15 1E
16 00 04
Found internal 48-bit s
Pairs from recovered se
#########################
1E 05 1E
11 1E 1D
1D 0F 0C
1D 0B 05

◄ 33m36s
```

43 bits

Brute force

```
1: /mnt/ram/BreakingLCG

java Gen
Generating 8 48-bit values from Random:
#################################
17 03 0D
0D 06 01
0A 15 1E
16 00 04
#################################
1E 05 1E
11 1E 1D
1D 0F 0C
1D 0B 05

  ram/BreakingLCG java U v21.0.8   23:58
```

```
cat gen.txt
17 03 0D
0D 06 01
0A 15 1E
16 00 04

  ram/Break
```

How to do it safely

```
import java.security.SecureRandom;

public class SecureRandomExample {
    public static void main(Stri
        SecureRandom rng = new SecureRandom();

        // Generate one number between 0 and 36 inclusive
        int value = rng.nextInt(37);  // upper bound is exclusive
        System.out.println("Random number: " + value);
    }
}
```

DO *NOT*

~~v = rng.nextInt() % 37;~~

~~seed ← System.nanoTime();~~

en.wik

ChaCha variant

In 2008, Bernstein pu

aim to increase the di

1: man getrandom

```
NOTES
       For an overview and comparison of the various interfaces t
hat can be used to obtain randomness, see random(7).

       Unlike /dev/random and /dev/urandom, getrandom() does not
involve the use of pathnames or file descriptors.  Thus, getrando
m() can be
```

WIKIPEDIA
The Free Encyclopedia

Donate Create account Log in

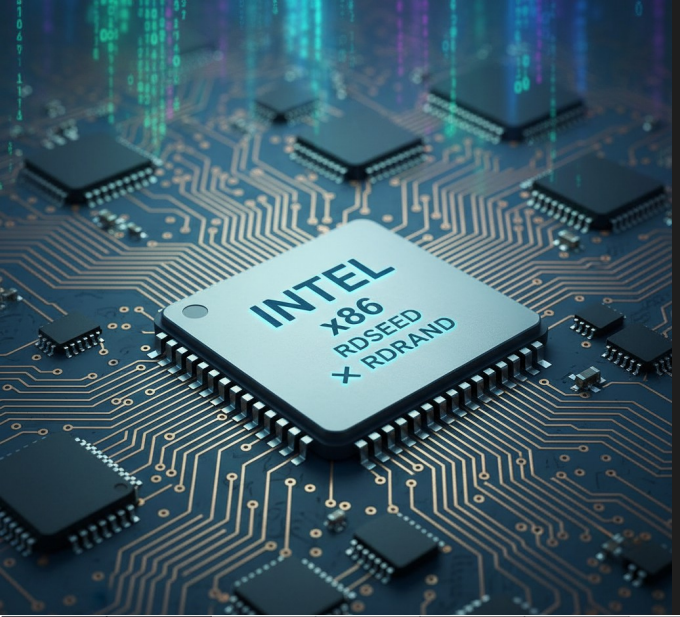
Cryptographically secure pseudorandom number generator

12 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

A **cryptographically secure pseudorandom number generator** (**CSPRNG**) or **cryptographic pseudorandom number generator** (**CPRNG**) is a pseudorandom number generator (PRNG) with properties that make it suitable for use in cryptography. It is also referred to as a **cryptographic random number generator** (**CRNG**).

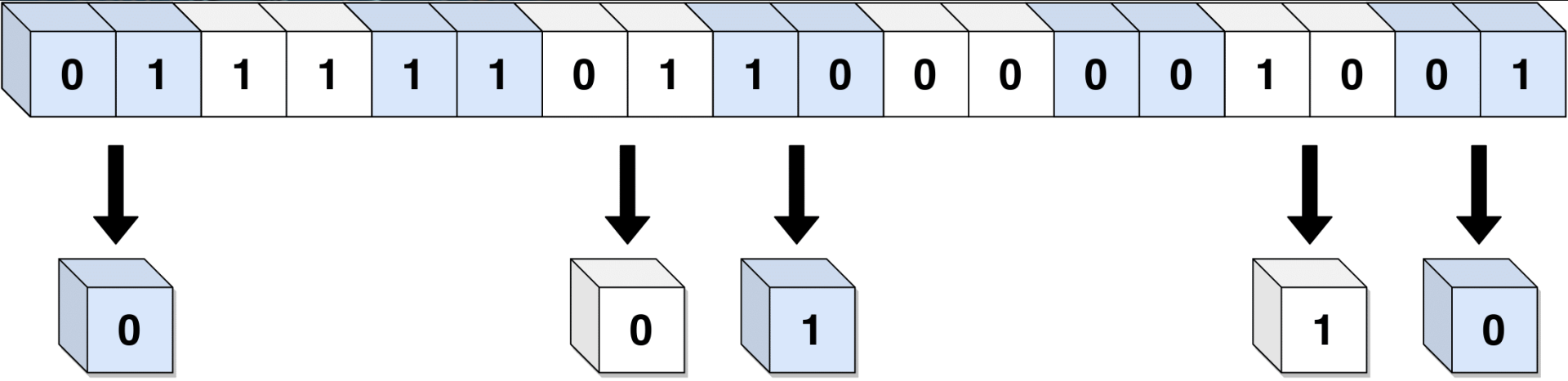


RDSEED / RDRAN

- Intel x86 hardware randomness
- may fail
- may not give desired throughput
- proprietary
- do not use directly

Von Neumann extractor

- remove bias (improves statistical quality)
- discard too many bits
- do not remove temporal correlation
- there are other newer better ones



Bad RNG incidents

- **Android Bitcoin wallets (2013)** — Android's `SecureRandom` implementation/usage produced weak ECDSA nonces (`k`). Reused or biased nonces let attackers recover private keys and drain wallets. Fix required updating Android and affected wallet apps. bitcoin.org +2
- **Sony PlayStation 3 (2010/2011)** — Firmware was signed with ECDSA but the nonce wasn't truly random (effectively reused). That exposed Sony's signing key, enabling counterfeit firmware. Schneier on Sec...
- **Debian OpenSSL bug (2006–2008)** — A Debian-specific patch gutted OpenSSL's entropy, making keys highly predictable (≈2^15 possibilities in some cases). SSH/SSL keys had to be regenerated. CVE-2008-0166. debian.org +1
- **Netscape SSL (1994–1996)** — PRNG seeded with predictable values (time, PID), so session keys were guessable; SSL connections could be decrypted. people.eecs.ber...
- **Dual_EC_DRBG (2007–2014)** — NIST-standardized DRBG later shown to permit a practical backdoor via special curve points; ultimately withdrawn by NIST. Some products (e.g., RSA BSAFE defaults; Juniper ScreenOS) were impacted. WIRED +3

What about v = r.nextInt() % 6

r (5 bits)	value = r % 6
0 - 5	0 1 2 3 4 5
6 - 11	0 1 2 3 4 5
12 - 17	0 1 2 3 4 5
18 - 23	0 1 2 3 4 5
24 - 29	0 1 2 3 4 5
30 - 31	0 1

dice	1 2	3 4 5 6
probability	6/32	5/32

```
int m = bound - 1;
if ((bound & m) == 0) {
    r = (int)((long)bound * (long)r >> 31);
} else {
    for(int u = r; u - (r = u % bound) + m < 0; u = this.next(31)) {
    }
}
```

What about v = r.nextInt() % 6 ?

r (32 bits)	value = r % 6
0 - 5	0 1 2 3 4 5
6 - 11	0 1 2 3 4 5
...	...
2^32-12 - 2^32-7	0 1 2 3 4 5
2^32-6 – 2^32-1	0 1 2 3

It is OK if it is negligible, but some apps may be not

- Cryptography
- Scientific Simulations
- Lottery/Gambling (under legal regulations)

```
1/6 * 4_294_967_292/4_294_967_296
= 0.16666666666666666667
```

```
+ 1/4 * 4/4_294_967_296
= 0.00000000023283064365
= 2.32 * 10^-10
```

Auditability

```
┌──────────┐                  ┌──────────┐
│  ALICE   │                  │   BOB    │
└──────────┘                  └──────────┘
      │                             │
(1) seed = Random()                 │
      │                             │
(2) hash = SHA256(seed)             │
      │                             │
(3) sig  = Sign(privKey, hash)      │
      │                             │
──────── Send {hash, sig, pubKey} ──────►│
                                    │
                        (4) Alice reveals seed
                                    │
                        (5) hash' = SHA256(seed)
                                    │
                        (6) Verify:
                            - hash' == hash ?
                            - Verify(pubKey, hash, sig) ?
                                    │
                                    ▼
                        ✅ If both true:
                           Integrity + Authenticity
                           (Non-repudiation ensured)
```

Stream **Cipher**

```
key = F1

 A  T  T  A  C  K
41 54 54 41 43 4B
F1 F1 F1 F1 F1 F1
B0 A5 A5 B0 B2 BA

 P  A  C  I  F  Y
50 41 43 49 46 59
F1 F1 F1 F1 F1 F1
A1 B0 B2 B8 B7 A8
```

```
PRNG(key) =
BB 19 68 68 35 09

 A  T  T  A  C  K
41 54 54 41 43 4B
BB 19 68 68 35 09
FA 4D 3C 29 76 42

 P  A  C  I  F  Y
50 41 43 49 46 59
BB 19 68 68 35 09
EB 58 2B 21 73 50
```

Enc

```
P = 0 1 1 0 1 0 0 1
K = 1 1 1 0 0 0 1 0
------- XOR -------
C = 1 0 0 0 1 0 1 1
```

Dec

```
C = 1 0 0 0 1 0 1 1
K = 1 1 1 0 0 0 1 0
------- XOR -------
P = 0 1 1 0 1 0 0 1
```

Could we use **LCG 512-bits for that ?**

Did you Enjoy ?

Let’s break it next time ?

Edgard Lima
edgard.lima@cesar.org.br

Thanks! :)